Informe Proyecto 3: Del Pochoco SoC a un macro ASIC

Arquitectura de Computadores · Semestre 2026-2 · Universidad de los Andes

Profesor: Jorge Gómez Mir

Integrantes: Lucas Fernández, Maximiliano Ferrando, Renato Rivadeneira y Matías Ureta

Repositorio: github.com/Max-Ferrando/Proyecto3_ASIC_Grupo7

Herramientas: LibreLane 3.0.16, GF180MCU (gf180mcuD), biblioteca gf180mcu_fd_sc_mcu7t5v0

1. Objetivo y resumen de resultados

En este proyecto adaptamos el Pochoco SoC del Proyecto 2, que ejecutaba el juego de reflejos en una FPGA, para implementarlo como un macro digital ASIC. El flujo se realizó con LibreLane y el PDK GF180MCU, desde el RTL hasta el archivo GDSII. El procesador y el programa se mantuvieron; los cambios principales se hicieron en la memoria, el reset y la interfaz física.

Comparamos dos memorias: A, con ROM combinacional y una RAM pequeña de flip-flops, y B, con memoria unificada de flip-flops. Según las corridas documentadas, ambas completaron el flujo con 100 ns de período y 40 % de utilización objetivo, sin errores de DRC, LVS o antena. Elegimos A porque ocupa menos área y permite un período menor.

En A, el die ocupa 0,444 mm² frente a 2,252 mm² en B. El menor período que cerró timing fue de 65 ns; al registrar la decodificación se redujo a 60 ns. También se probó una utilización de 62 %, que disminuyó el área del die a 0,291 mm². Las simulaciones RTL y post-síntesis mostraron las mismas salidas para los estímulos utilizados.

1.1. Guía de contenido

Tema	Sección
Adaptación del RTL de FPGA a ASIC	2
Opciones de memoria y mediciones	3
Síntesis y área por módulo	4
Timing, ruta crítica y mejora	5
Floorplan, utilización y costo	6
Signoff y layout	7
Simulación post-síntesis	8
Versiones y reproducción	9

2. Adaptación del diseño a ASIC

2.1. Organización del RTL

Para no mezclar las dependencias de la FPGA con el nuevo diseño, dejamos el procesador y los periféricos compartidos en rtl/common, lo específico de la Go Board en rtl/fpga y las implementaciones de memoria, banco de registros y top-level ASIC en rtl/asic. El módulo asic_top tiene puertos lógicos para reloj, reset, botones, LEDs, displays y SPI. No incluye pads físicos ni circuitos de protección.

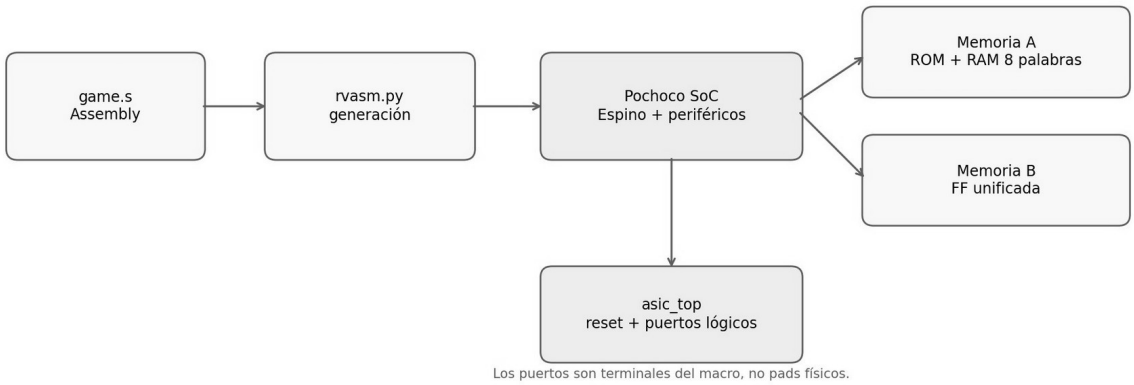


Figura 1. Flujo entre el assembler, el SoC y las dos alternativas de memoria.

Archivo o módulo	Función
asic_top.v	Módulo superior del macro ASIC.
pochoco_reset_sync.v	Reset activo en bajo con liberación sincronizada por dos flip-flops.
pochoco_soc.v	Interconecta el procesador, la memoria y los periféricos.
espino_register_file_ff.v	Banco de registros RV32E implementado con FF y lectura registrada.
pochoco_cycle_counter.v	Contador de 32 bits accesible desde el programa.
pochoco_mem_rom.v / pochoco_mem_ff.v	Alternativas A y B de memoria.
gen/game_rom.v / gen/game_mem.vh	Archivos generados por el assembler a partir de game.s.

2.2. Problemas encontrados al pasar de FPGA a ASIC

Como primera prueba intentamos sintetizar el RTL del Proyecto 2 con LibreLane sin adaptarlo. Luego hicimos otra corrida sustituyendo el banco de registros original. Esto permitió identificar qué construcciones impedían la síntesis y cuáles eran peligrosas aunque no produjeran un error.

Construcción anterior	Evidencia obtenida	Cambio aplicado
Primitiva SB_RAM40_4K	Verilator: MODMISSING en espino_register_file.v:22 y :33; Yosys: módulo SB_RAM40_4K ausente, jsonheader.log:474.	Banco de registros nuevo con FF y multiplexores.
Registro por_cnt con valor inicial	Yosys synthesis.log:283 registra init=0; después se mapea a FF sin reset (línea 980).	Entrada rst_ni y sincronizador de reset.
$readmemh y BRAM de 512 palabras	Sin error. stat.rpt muestra 16.452 dffq_1 y 16.459 mux2_2; área 2.326.841,75 µm².	Generación de ROM lógica o memoria de FF con inicialización de reset.
Frecuencia fija en game.s	No emite error: es una suposición del software.	CLK_HZ actualizado de 25 MHz a 10 MHz.
Display activo en bajo	No emite error: depende de la placa.	La inversión de polaridad queda en el top de FPGA.



Figura 2. Error de la primitiva de memoria de la iCE40 en el RTL original.

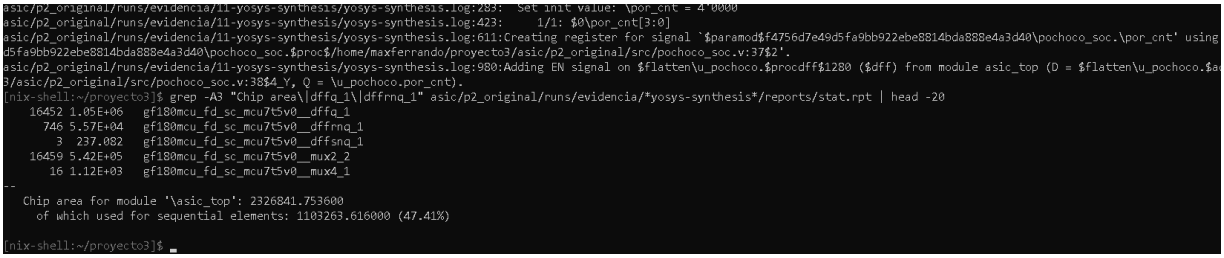


Figura 3. Avisos de inicialización y costo de implementar la memoria original con FF.

La principal diferencia es que una FPGA puede cargar ciertos contenidos al programarse, pero un ASIC no. Por eso no basta con que Yosys acepte $readmemh o un valor inicial: debe existir una solución física para que el programa y el estado de reset estén definidos al encender.

3. Memoria y programa del juego

3.1. Cuánta memoria utiliza el programa

El assembler reportó 153 palabras de instrucciones (612 bytes) para la configuración de 10 MHz. Las 24 instrucciones lw/sw del juego usan como base la dirección de periféricos 0x8000_0000. En simulación, el fetch llegó como máximo a 0x258 en la variante reducida y no se registraron accesos a la RAM de datos.

Por lo tanto, el juego necesita 153 palabras de instrucciones y no usa RAM de datos de propósito general. De todos modos reservamos ocho palabras de 32 bits para permitir que el sistema ejecute otros programas con variables o pila. La ROM tiene espacio para 256 instrucciones.

3.2. Estrategias implementadas



Figura 4. Esquema de la memoria de instrucciones y datos en las dos alternativas.

Característica	A: ROM + RAM de FF	B: memoria unificada de FF
Programa	ROM combinacional desde game_rom.v	Contenido inicial en FF mediante reset
Instrucciones	153 palabras, capacidad 256	153 palabras en memoria unificada
Datos reservados	8 palabras desde 0x0000_0400	8 palabras en memoria unificada
Lectura	ROM con salida registrada	Dos puertos de lectura registrados
Software	Mismo game.s	Mismo game.s
Limitación	El puerto de datos no lee ROM	Cada bit del programa requiere un FF

3.3. Comparación de resultados

La tabla resume las corridas finales con CLOCK_PERIOD = 100 ns y FP_CORE_UTIL = 40 %. El período mínimo de B se estimó desde el margen a 100 ns; no corresponde a un barrido completo como el de A.

Métrica	Estrategia A	Estrategia B
Área del die	444.189 µm²	2.252.425 µm²
Área de celdas	213.569 µm²	1.347.150 µm²
Flip-flops	1.070	5.967
Utilización real	51,58 %	61,66 %
Setup WS típico / lento	+65,917 / +35,267 ns	+54,196 / +11,329 ns
Hold WS típico / lento	+0,408 / +0,996 ns	+0,324 / +0,822 ns
Período mínimo	65 ns, medido	~89 ns, estimado
Frecuencia máxima	15,4 MHz, medida	~11,3 MHz, estimada
Tiempo de corrida	5 min 17 s	34 min 55 s
Costo equivalente	US$10.216	US$51.806

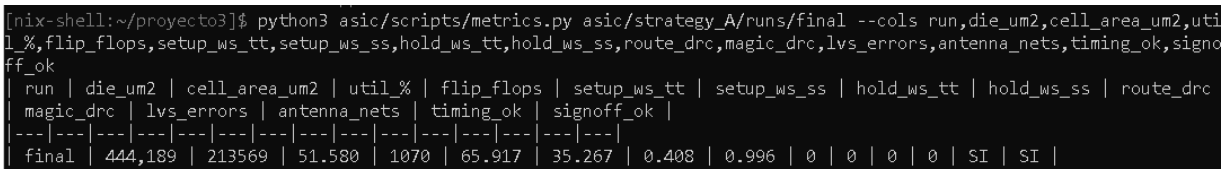


Figura 5. Salida de métricas de la estrategia A.

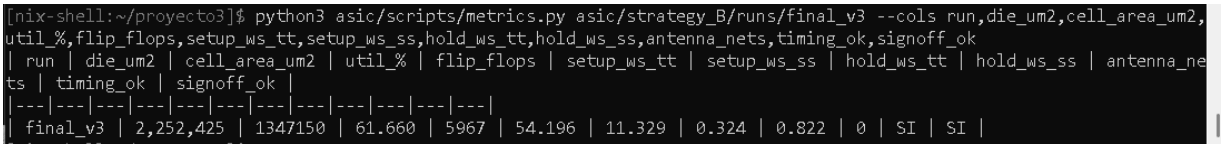


Figura 6. Salida de métricas de la estrategia B.

B ocupa unas 5,1 veces el área de A. La ROM combinacional se optimiza como lógica, mientras que la memoria unificada necesita FF por bit y multiplexores grandes para los puertos de lectura. También aparecieron redes de alto fanout; una de ellas tenía 1.002 terminales. Por área y timing, continuamos el resto de las pruebas con A.

4. Síntesis y área por módulo

LibreLane aplana parte de la jerarquía. Para seguir identificando el banco de registros, la ALU y el contador, configuramos SYNTH_KEEP_HIERARCHY_MODULES para espino_register_file, espino_alu y pochoco_cycle_counter. Para estimar el área por módulo también utilizamos area_by_module.py, que vuelve a sintetizar manteniendo la jerarquía.

4.1. Celdas más utilizadas

Celda	Cantidad	Principal motivo de uso
mux2_1	994	Retención en el banco de registros y selección de operandos.
dffrnq_1	812	FF con reset: banco, registros del pipeline, PC y contador.
nand2_1	649	Árboles de multiplexores de lectura y decodificación de direcciones.
aoi22_1	463	Lógica de selección del banco y ROM.
dffq_1	256	RAM de datos de 8 × 32 bits, sin reset.

4.2. Área por bloque antes del aplanamiento

Bloque	Celdas	Área (µm²)	Porcentaje
Banco de registros	1.796	69.400	42,6 %

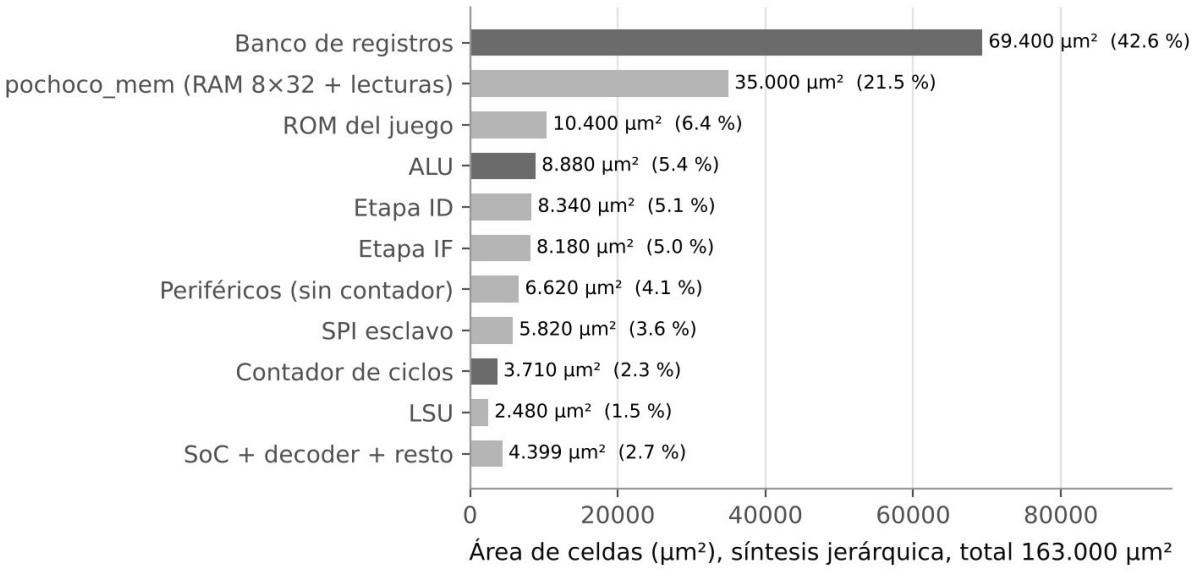


Figura 7. Distribución del área estimada por módulo en síntesis jerárquica.

```
[nix-shell:~/proyecto3]$ python3 asic/scripts/area_by_module.py asic/strategy_A/runs/final
Sintetizando con jerarquia (16 archivos, T = 100.0 ns)...
Warning: Replacing memory \regs with list of registers. See rtl/asic/espino_register_file_ff.v:34

Area por modulo (incluye submodulos):
  asic_top                                          5329 celdas      163,000 um2  100.0 %
  pochoco_reset_sync                                   2 celdas          149 um2    0.1 %
  pochoco_soc                                         76 celdas        2,150 um2    1.3 %
  $paramod\espino_if_stage\BootAddr=32'00000000000000000000000000000000     226 celdas        8,180 um2    5.0 %
  espino_controller                                    5 celdas           70 um2    0.0 %
  espino_id_stage                                    415 celdas        8,340 um2    5.1 %
  espino_alu                                         539 celdas        8,880 um2    5.4 %
  espino_decoder                                     140 celdas        2,030 um2    1.2 %
  espino_load_store_unit                             148 celdas        2,480 um2    1.5 %
  espino_register_file                              1796 celdas       69,400 um2   42.6 %
  pochoco_mem                                        895 celdas       35,000 um2   21.5 %
  pochoco_game_rom                                   614 celdas       10,400 um2    6.4 %
  pochoco_periph                                     232 celdas        6,620 um2    4.1 %
  pochoco_cycle_counter                               97 celdas        3,710 um2    2.3 %
  pochoco_spi_slave                                  144 celdas        5,820 um2    3.6 %

Cinco celdas mas usadas:
     994  mux2_1
     812  dffrnq_1
     649  nand2_1
     463  aoi22_1
     256  dffq_1

Reporte completo: asic/strategy_A/runs/final/area_by_module.rpt
[nix-shell:~/proyecto3]$
```

Figura 8. Salida del script area_by_module.py.

Como esperábamos, el banco de registros es el bloque individual más grande. Tiene 480 FF para x1-x15, multiplexores para habilitar las escrituras y dos puertos de lectura. El contador representa solo 2,3 % de esta estimación. El área jerárquica (163.000 µm²) es menor que el área final de celdas (213.569 µm²), ya que el flujo físico agrega buffers, reparaciones de timing y otras celdas. En el layout final se identificaron 2.191 celdas del banco, con un área de 112.109 µm².

5. Temporización y ruta crítica

5.1. Barrido de CLOCK_PERIOD

La configuración requerida es de 100 ns, equivalente a 10 MHz. Consideramos que una corrida cierra timing solo si setup y hold tienen WNS no negativo y TNS igual a cero en todos los corners exigidos. La tabla muestra el típico y el lento (SS, 125 °C, 4,5 V).

T (ns)	Setup típico WS / TNS	Hold típico WS / TNS	Setup lento WS / TNS	Hold lento WS / TNS	Cierra
100	+65,917 / 0	+0,408 / 0	+35,267 / 0	+0,996 / 0	Sí
70	+35,917 / 0	+0,408 / 0	+5,267 / 0	+0,996 / 0	Sí
65	+30,917 / 0	+0,408 / 0	+0,267 / 0	+0,996 / 0	Sí
60	+25,917 / 0	+0,408 / 0	-4,733 / -154,605	+0,996 / 0	No
50	+17,322 / 0	+0,408 / 0	-11,825 / -4.167	+0,997 / 0	No

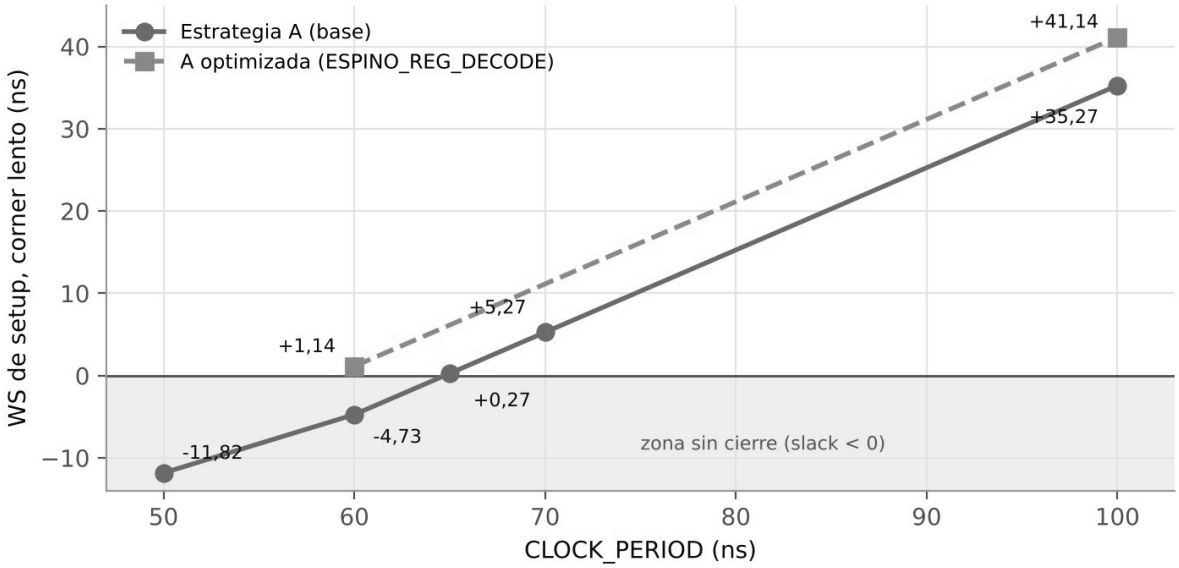


Figura 9. Slack en el corner lento para la versión base y la optimizada.

La menor configuración que cerró para A fue 65 ns, equivalente a 15,4 MHz. A 60 ns ya aparecen violaciones en el corner lento. El área de celdas apenas cambió durante el barrido, por lo que acortar el período no provocó una optimización suficiente para cerrar ese corner.

```
<<< CLOCK_PERIOD_50: OK en 5.7 min

| run | CLOCK_PERIOD_ns | FP_CORE_UTIL_% | die_um2 | cell_area_um2 | util_% | setup_ws_tt | setup_tns_tt | setup_ws_ss | setup_tns_ss | hold_ws_tt | hold_tns_tt | hold_ws_ss | hold_tns_ss | route_drc | grt_overflow | antenna_nets | timing_ok | signoff_ok |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| CLOCK_PERIOD_70 | 70 | 40 | 444,189 | 213569 | 51.580 | 35.917 | 0 | 5.267 | 0 | 0.408 | 0 | 0.996 | 0 | 0 | 64 | 0 | SI | SI |
| CLOCK_PERIOD_60 | 60 | 40 | 444,189 | 213569 | 51.580 | 25.917 | 0 | -4.733 | -154.605 | 0.408 | 0 | 0.996 | 0 | 0 | 64 | 0 | NO | SI |
| CLOCK_PERIOD_50 | 50 | 40 | 444,189 | 213547 | 51.580 | 17.322 | 0 | -11.825 | -4,167 | 0.408 | 0 | 0.997 | 0 | 0 | 69 | 0 | NO | SI |

Tabla completa: /home/maxferrando/proyecto3/asic/strategy_A/sweep_CLOCK_PERIOD.csv
  CLOCK_PERIOD_70: codigo 0, 5.7 min
  CLOCK_PERIOD_60: codigo 0, 5.6 min
  CLOCK_PERIOD_50: codigo 0, 5.7 min
[nix-shell:~/proyecto3]$
```

Figura 10. Corridas a 70, 60 y 50 ns.

```
[nix-shell:~/proyecto3]$ python3 asic/scripts/metrics.py asic/strategy_A/runs/CLOCK_PERIOD_65 --cols run,setup_ws_tt,setup_tns_tt,setup_ws_ss,setup_tns_ss,hold_ws_tt,hold_ws_ss,timing_ok
| run | setup_ws_tt | setup_tns_tt | setup_ws_ss | setup_tns_ss | hold_ws_tt | hold_ws_ss | timing_ok |
|---|---|---|---|---|---|---|---|
| CLOCK_PERIOD_65 | 30.917 | 0 | 0.267 | 0 | 0.408 | 0.996 | SI |
[nix-shell:~/proyecto3]$
```

Figura 11. Corrida a 65 ns, sin violaciones de timing.

5.2. Ruta crítica

La ruta crítica se extrajo del reporte STA de la corrida final A (max_ss_125C_4v50). Parte en el registro de instrucción u_soc.instr_rdata[3] y termina en u_soc.mem_b_rdata[22], registro de lectura de memoria. Cruza fetch, decodificación, ALU y selección de dirección de memoria, con una llegada de 66,41 ns y slack de +35,27 ns a 100 ns.

Etapa	Tiempo acumulado	Detalle
Reloj y clk→Q	0 a 5,14 ns	Salida del registro de instrucción.
Decodificación	5,14 a 9,45 ns	Lógica nand/nor de control.

Etapa	Tiempo acumulado	Detalle
Fanout	9,45 a 29,59 ns	Buffers y selección de operandos.
ALU	29,59 a 44,07 ns	Sumador hasta lsu_addr[31].
Selección de memoria	44,07 a 66,41 ns	Selección RAM/periféricos y registro de lectura.

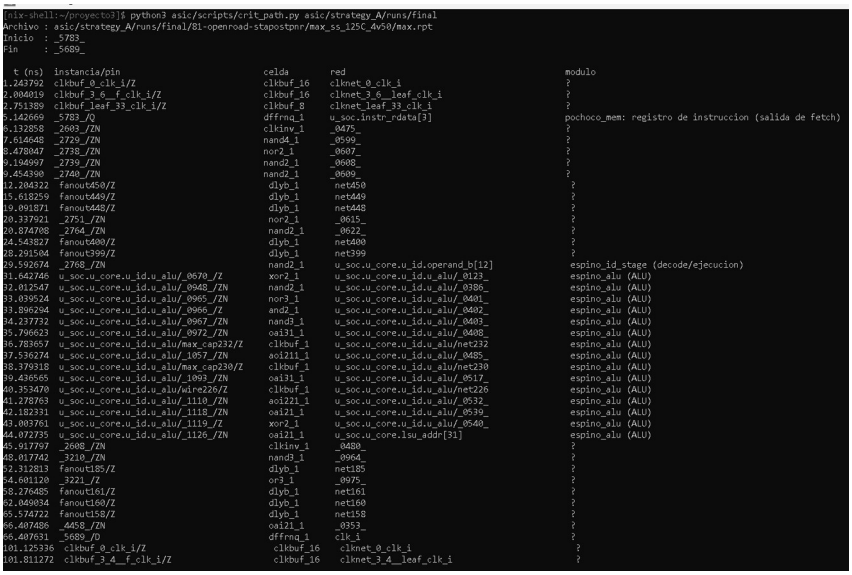


Figura 12. Ruta crítica identificada en el reporte del corner lento.

Un detalle que llamó la atención fue que nueve celdas dlyb_1 aportaban cerca de 30 ns a esta ruta. Se usaron como buffers de fanout, pero son celdas de retardo. Por eso una mejora posterior sería evitar ese tipo de celda en esa red o registrar la dirección de memoria.

5.3. Cambio evaluado en el RTL

Probamos ESPINO_REG_DECODE. Como la lectura del banco está registrada, hay un ciclo de espera en decode; en ese ciclo registramos también las señales de decodificación. Con esto el decoder deja de estar en la ruta combinacional hacia la ALU, sin cambiar la cantidad de ciclos del juego.

Métrica	A base	A optimizada
Área de celdas	213.569 µm²	221.256 µm² (+3,6 %)
Área del die	444.189 µm²	458.601 µm² (+3,2 %)
Setup WS lento a 100 ns	+35,267 ns	+41,141 ns
Hold WS lento	+0,996 ns	+0,984 ns
Menor período cerrado	65 ns	60 ns
Frecuencia máxima	15,4 MHz	16,7 MHz

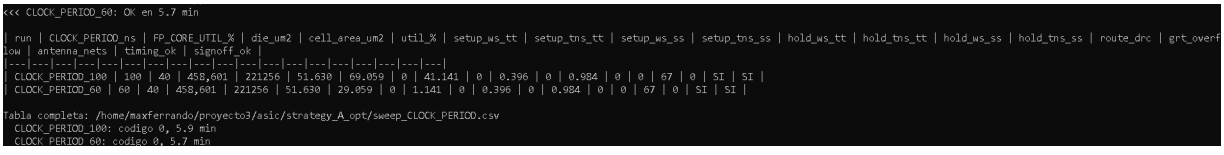


Figura 13. Cierre de timing de la variante optimizada a 60 ns.

La variante optimizada alcanza unos 16,7 MHz, aproximadamente 8 % más que la base, a cambio de 3,6 % adicional de área de celdas. No cambia la parte más larga de la ruta de acceso a memoria, por eso la mejora tiene un límite. En la simulación, los casos medidos mantuvieron 14.349 y 8.778 ciclos.

5.4. Constantes del programa para 10 MHz

Al cambiar el reloj de 25 MHz en FPGA a 10 MHz en el ASIC, actualizamos CLK_HZ y regeneramos el programa. El resto de las constantes se calcula a partir de la frecuencia:

Constante	FPGA 25 MHz	ASIC 10 MHz
CLK_HZ	25.000.000	10.000.000
TENTH	2.500.000	1.000.000
HALF_TENTH	1.250.000	500.000
ALL_ON_TIME	75.000.000	30.000.000
DEBOUNCE	500.000	200.000
MAX_CYCLES	247.500.000	99.000.000
Divisor del promedio	25.000.000	10.000.000

Así, tres segundos corresponden a 30.000.000 ciclos y cada décima a 1.000.000. Diez rondas de hasta 9,9 s acumulan como máximo 990.000.000 ciclos, por debajo del límite de 32 bits. El contador completo tarda aproximadamente 429,5 s en dar la vuelta a 10 MHz.

6. Floorplan, terminales y costo

Con la estrategia A realizamos corridas a 40, 55, 62 y 70 % de utilización objetivo. Para elegir la mejor no consideramos solo el área: también debía terminar el ruteo, cerrar timing y mantener el signoff limpio.

FP_CORE_UTIL	Área die (µm²)	Utilización real	Congestión / ruteo	Setup WS lento	Resultado
40 %	444.189	51,58 %	Overflow 64; DRC 0	+35,267 ns	Cumple
55 %	326.784	69,74 %	Overflow 39; DRC 0	+40,566 ns	Cumple
62 %	291.282	78,85 %	Overflow 98; DRC 0	+40,460 ns	Cumple
70 %	—	—	66.177 → 3.867 violaciones	—	No cierra

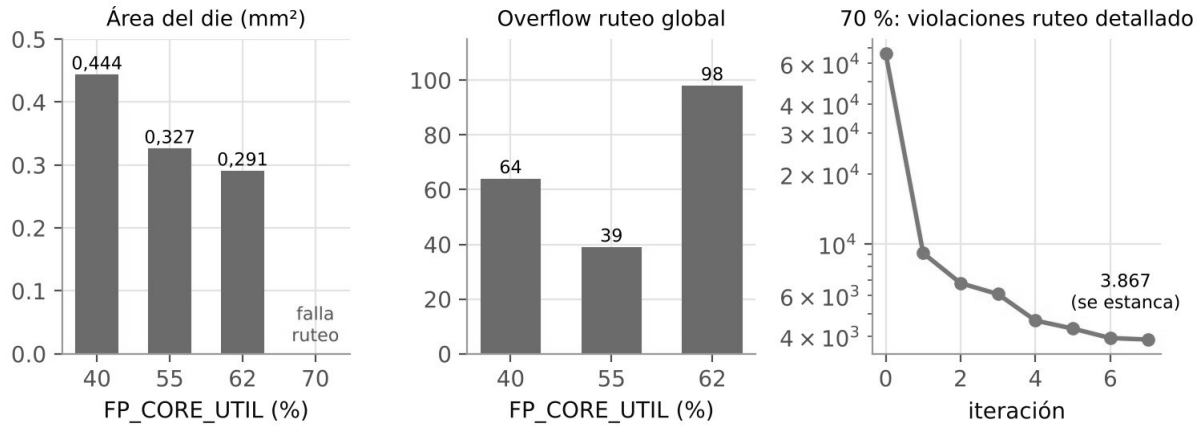


Figura 14. Área, congestión y violaciones de ruteo según la utilización objetivo.

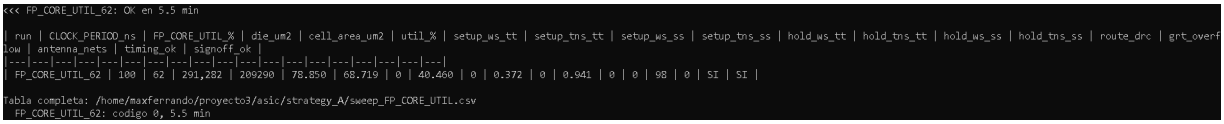


Figura 15. Resultado de la corrida a 62 %.

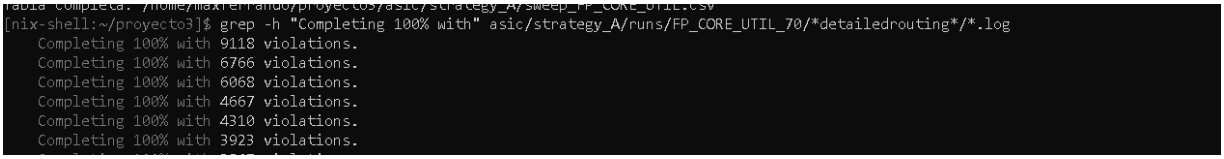


Figura 16. Violaciones de ruteo al intentar 70 %.

El menor die con flujo completo fue el de 62 %: 0,291 mm², un 34 % menos que el de 40 %. A 70 % el ruteo detallado no pudo resolver las violaciones. El aumento de utilización no empeoró linealmente el timing, porque cambió el largo de las conexiones y el trabajo del ruteador.

6.1. Ubicación de terminales

Se agruparon los terminales por función en asic/pin_order.cfg: reloj y reset al oeste, botones al norte, LEDs y displays al sur, SPI al este. Son terminales de un macro, no pads de entrada/salida de un chip terminado.

Lado	Señales	Motivo
Oeste	clk_i, rst_ni	Reloj y reset separados.
Norte	btn_i.*	Entradas de botones agrupadas.
Sur	led_o.*, seg1_o.*, seg2_o.*	Salidas visuales.
Este	spi_*	Interfaz de comunicación.

6.2. Estimación de costo

Para el ejercicio usamos la relación lineal indicada en clases: US$23.000 por cada mm² de área del die. Es una estimación didáctica; no contempla rendimiento de fabricación, costos fijos, pads ni encapsulado.

Diseño	Área (mm²)	Costo equivalente
A con 40 %	0,444	US$10.216
A con 62 %	0,291	US$6.699
A optimizada con 40 %	0,459	US$10.548
B con 40 %	2,252	US$51.806
Proyecto 2 sin adaptar (solo celdas)	≥ 2,327	≥ US$53.500

La decisión con más efecto en el costo fue la memoria: B cuesta unas 5,1 veces más que A bajo este supuesto. Después influyó la utilización: al aumentar de 40 a 62 %, el costo equivalente de A bajó alrededor de 34 %.

7. Signoff y layout

Revisión	Estrategia A	Estrategia B
DRC de Magic	0 errores	0 errores
DRC de ruteo	0	0
LVS de Netgen	0 errores	0 errores
Antena	0 redes	0 redes en final_v3
Setup y hold	Cierra en corners requeridos	Cierra en corners requeridos
GDSII	final/gds/asic_top.gds	final/gds/asic_top.gds

Para gf180mcuD, LibreLane no ejecutó el DRC de KLayout porque faltaba KLAYOUT_DRC_RUNSET. El DRC de signoff documentado es el de Magic. También aparece GRT-0281 por el fanout del reloj antes de construir el árbol de clock.



Figura 17. Resumen de signoff de la estrategia A.

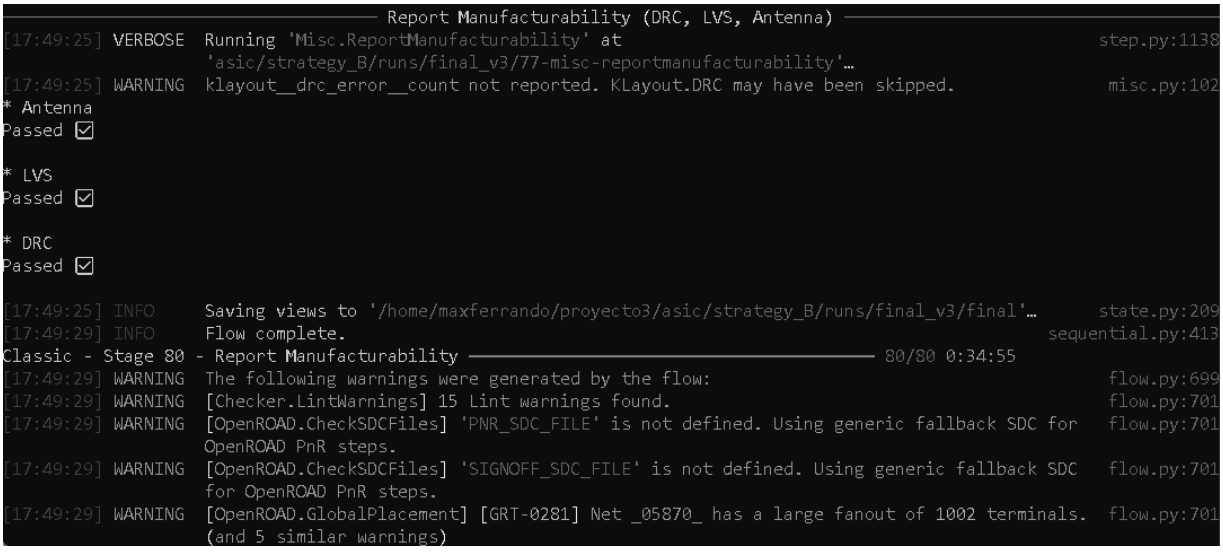


Figura 18. Resumen de signoff de la estrategia B final_v3.

7.1. Ubicación del banco, la ALU y el contador

El script highlight_layout.py identifica las celdas en el DEF a partir de los prefijos jerárquicos y marca su posición sobre el GDS para revisarlo en KLayout. El banco ocupa gran parte del lado izquierdo; la ALU está cerca del banco y el contador aparece hacia la derecha.

Bloque	Celdas finales	Área (µm²)	Rectángulo aproximado (µm)
Banco de registros	2.191	112.109	(8, 39) a (345, 608)
ALU	615	15.843	(254, 212) a (428, 420)
Contador de ciclos	117	5.390	(494, 278) a (635, 448)

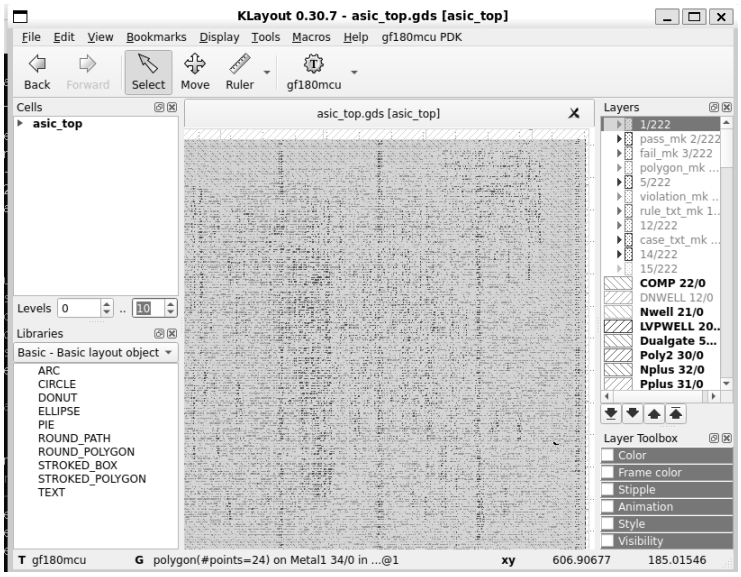


Figura 19. GDSII final abierto en KLayout.

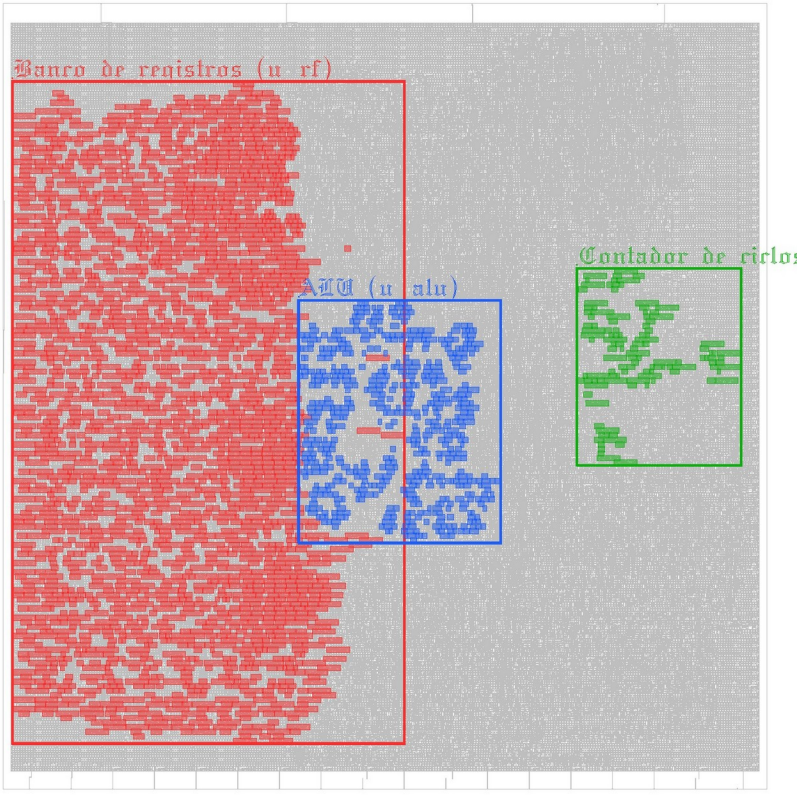


Figura 20. Layout con el banco de registros, la ALU y el contador identificados.

7.2. Problema de antena en la estrategia B

En B apareció una violación de antena en una pista de Metal2 que llegaba a una compuerta de la red _05410_. Primero aumentamos la reparación de antena del ruteo global, pero no bastó. La última corrida habilitó también la reparación en ruteo detallado y la inserción heurística de diodos.

Corrida	Cambio	Resultado
final	Configuración base	1 red con violación.
final_v2	Más iteraciones y margen en GRT	La red continuó fallando.

Corrida	Cambio	Resultado
final_v3	Reparación en DRT y diodos heurísticos	0 violaciones; signoff aprobado.

```
[nix-shell:~/proyecto3]$ grep -E "ANTENNA_REPAIR|DIODE_ON_PORTS" asic/strategy_B/runs/final_v2/resolved.json
    "RUN_ANTENNA_REPAIR": true,
    "GRT_ANTENNA_REPAIR_ITERS": 10,
    "GRT_ANTENNA_REPAIR_MARGIN": 20,
    "GRT_ANTENNA_REPAIR_JUMPER_ONLY": false,
    "GRT_ANTENNA_REPAIR_DIODE_ONLY": false,
    "DIODE_ON_PORTS": "in",
    "DRT_ANTENNA_REPAIR_ITERS": 3,
    "DRT_ANTENNA_REPAIR_MARGIN": 10,
    "DRT_ANTENNA_REPAIR_JUMPER_ONLY": false,
    "DRT_ANTENNA_REPAIR_DIODE_ONLY": false,
[nix-shell:~/proyecto3]$ cat asic/strategy_B/runs/final_v2/*checkantennas*/reports/*.rpt | head -40
Net: _05410_
  Pin:   fanout4273/I (gf180mcu_fd_sc_mcu7t5v0__dlyb_1)
    Layer: Via1
      Partial area ratio:    0.17
      Required ratio:   20.00 (Gate area)
      Cumulative area ratio:    0.17
      Required ratio:    0.00 (Cumulative area)

    Layer: Metal2
      Partial area ratio:    3.93
      Required ratio:    0.00 (Gate area)
      Cumulative area ratio:    3.93
      Required ratio:    0.00 (Cumulative area)
      Partial area ratio:   15.30
```

Figura 21. Reporte y configuración utilizados para la reparación de antena.

Durante la última corrida B también hubo un cierre de STA por falta de memoria en WSL. Se aumentó la memoria a 5 GB, el swap a 8 GB y se limitó el número de trabajos a dos. En A no aparecieron violaciones de antena.

8. Simulación funcional post-síntesis

Usamos sim/asic/tb_asic.v y un archivo común de estímulos con el ciclo de cada pulsación. Se simularon el RTL y el netlist de la estrategia A con modelos Verilog de GF180MCU. Para hacer la simulación más rápida, ambos usaron el programa ensamblado con CLK_HZ = 20.000; esto deja una décima en 2.000 ciclos y la espera de 3 segundos en 60.000 ciclos.

Prueba	RTL	Netlist A
Primera respuesta correcta	14.349 ciclos → 07	14.349 ciclos → 07
Respuesta incorrecta	EE; 6.006 ciclos	EE; 6.006 ciclos
Reinicio de la ronda	Conserva 1 acierto	Conserva 1 acierto
Tercera pulsación correcta	8.778 ciclos → 04	8.778 ciclos → 04
Final	2 rondas correctas, 04	2 rondas correctas, 04
Trazas de LEDs y displays	Referencia RTL	Coinciden ciclo a ciclo



Figura 22. Comparación de la traza RTL con la del netlist post-síntesis.

En estas pruebas, la síntesis conservó la respuesta temporal observada en RTL: una pulsación incorrecta muestra EE y repite la ronda, pero no elimina las correctas anteriores. La variante B también entregó los mismos resultados funcionales a nivel RTL.

9. Versiones y reproducibilidad

Las versiones usadas para las corridas del informe son las siguientes:

Herramienta / ajuste	Versión o valor
PDK	gf180mcuD, revisión 54435919abffb937387ec956209f9cf5fd2dfbee (2025.12.26)
Biblioteca estándar	gf180mcu_fd_sc_mcu7t5v0
CLOCK_PERIOD / FP_CORE_UTIL	100 ns / 40 %
Top-level	asic_top
LibreLane	3.0.16 (WSL2 Ubuntu / nix-shell)
Yosys	0.62 (git 7326bb7d)
OpenROAD	dcf36133a369abc8f3c5e5738cd4d82e4903c0e0
Magic / Netgen	8.3.623 / 1.5.316
KLayout / Verilator	0.30.7 / 5.044

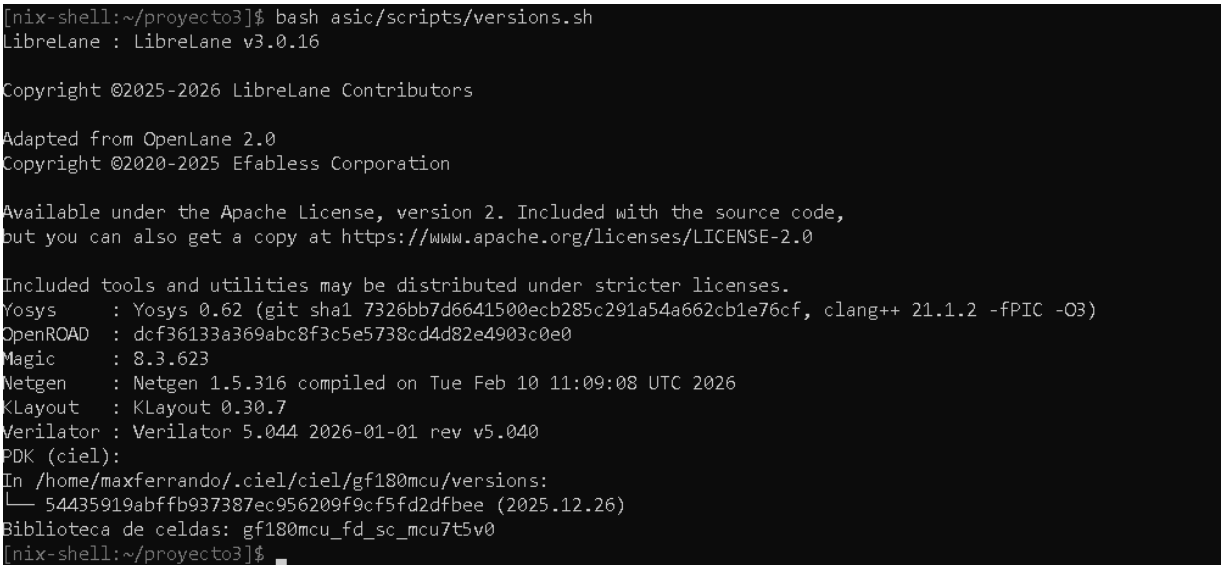


Figura 23. Salida del script de versiones del entorno usado.

Desde la raíz del repositorio, dentro de nix-shell, los comandos principales para repetir las corridas son:

```
make asic-gen
make asic-A asic-B asic-A-opt
make asic-sim asic-synth-sim
python3 asic/scripts/sweep.py asic/strategy_A/config.yaml CLOCK_PERIOD 70 65 60 50
python3 asic/scripts/sweep.py asic/strategy_A/config.yaml FP_CORE_UTIL 55 62 70
python3 asic/scripts/metrics.py asic/strategy_A/runs/*
python3 asic/scripts/crit_path.py asic/strategy_A/runs/final
python3 asic/scripts/area_by_module.py asic/strategy_A/runs/final
python3 asic/scripts/highlight_layout.py asic/strategy_A/runs/final
bash asic/scripts/versions.sh
```

10. Conclusiones

El paso de FPGA a ASIC no exigió rehacer Espino Core, pero sí cambiar varios supuestos del diseño original. Las primitivas de memoria del iCE40 no existen en la biblioteca GF180 y los valores iniciales que carga una FPGA no aparecen automáticamente al encender un chip. Por eso se implementaron memoria y reset específicos para ASIC.

La decisión que más influyó en el resultado fue cómo guardar el programa: la ROM combinacional con RAM pequeña usa bastante menos área que la memoria unificada de flip-flops. El banco de registros sigue siendo una parte importante del costo del procesador, mientras que el contador agregado para el juego ocupa una fracción pequeña.

Con los parámetros obligatorios de 100 ns y 40 % logramos las comprobaciones de timing y signoff que se muestran en el informe. También encontramos dos mejoras concretas: registrar la decodificación para alcanzar 60 ns, y aumentar la utilización objetivo a 62 % para reducir el die sin perder cierre de flujo. El intento de 70 % no pudo completar el ruteo.

Las pruebas post-síntesis fueron útiles para confirmar que las salidas y los tiempos del juego se conservaron. Como posibles mejoras quedan eliminar la RAM de datos que este programa no usa, estudiar el alto fanout de algunas redes y evitar las celdas de retardo que aparecen en la ruta crítica.

Anexo A. Archivos principales del repositorio

Ruta	Uso
asic/strategy_A/config.yaml	ROM y RAM, configuración base.
asic/strategy_B/config.yaml	Memoria unificada de FF.
asic/strategy_A_opt/config.yaml	Variante de decode registrado.
asic/p2_original/	RTL original y corridas de incompatibilidad.
asic/pin_order.cfg / pin_order.md	Terminales del macro y justificación.
asic/scripts/sweep.py y metrics.py	Barridos y resumen de resultados.
asic/scripts/crit_path.py	Reporte de la ruta crítica.
asic/scripts/area_by_module.py	Área por módulo.
asic/scripts/highlight_layout.py	Ubicación de bloques en el GDS.
sim/asic/tb_asic.v, sim/asic/stim.txt, sim.sh	Simulación RTL y post-síntesis.
tools/rvasm.py	Generación de ROM y memoria desde el programa.

Anexo B. Incidentes registrados

Problema	Qué se hizo
PDK gf180mcuD no encontrado	Se pasó --pdk gf180mcuD al invocar LibreLane.
Error con # en pin_order.cfg	Se quitaron comentarios del archivo y se explicó el orden en pin_order.md.
Antena en memoria B	Más reparación en ruteo detallado e inserción de diodos.
SIGKILL en STA de B	Se aumentó memoria y swap de WSL, con --jobs 2.
70 % no terminó de rutear	Se seleccionó 62 % como máximo válido probado.
sim.sh: Permission denied	Se corrigió el permiso de ejecución del script.